FilmSnob: Personalized Movie Recommendations from Letterboxd Reviews

Rayan Singh
University of Illinois
Urbana-Champaign

Ryan Vir
University of Illinois
Urbana-Champaign

Tony Chan
University of Illinois
Urbana-Champaign

ABSTRACT

1 INTRODUCTION

2 SYSTEM OVERVIEW

FilmSnob is a content-based movie recommender. We scrape a user's Letterboxd diary (what they watched, how they rated it, what they wrote about it), pull in extra metadata from The Movie Database (TMDB), and build a vector for each film that combines sentence-transformer output with structured features like genre and year. PCA brings the dimensionality down to something manageable, and we recommend films by finding the closest unwatched movies to the user's taste profile via cosine similarity.

The pipeline has four stages:

(1) **Letterboxd ingestion:** pull user film diaries via RSS, HTML scraping, or CSV export.
(2) **TMDB enrichment:** look up plot summaries, genres, cast, director, and keywords through the TMDB API.
(3) **Cross-user aggregation:** merge per-user data into one film corpus, pooling all reviews for each film.
(4) **Embedding and recommendation:** encode films with SBERT + metadata, reduce with PCA, and run nearest-neighbor search.

3 DATA COLLECTION

3.1 Letterboxd Ingestion

We support three ways to get data out of Letterboxd, all of which produce the same internal `UserProfile` object:

RSS feed (primary). Every Letterboxd user has a public RSS feed at `/{username}/rss/`. It gives back up to 50 recent diary entries as XML, with custom `letterboxd:` and `tmdb:` namespaces. Each entry has the film title, year, the user's star rating (0.5–5.0), watch date, and the TMDB movie ID. Review text shows up in the `<description>` field as HTML, which we strip out with BeautifulSoup.

We went with RSS over HTML scraping mainly because Letterboxd puts most of its pages behind Cloudflare (we kept hitting HTTP 403s during development). RSS feeds are not blocked. As a bonus, they include the TMDB movie ID directly, so we can skip the title-based search step when looking up metadata.

HTML scraping (fallback). We parse the poster grid at each user's films page for slugs and ratings, then their reviews page for review text. This can get a full watch history, but Cloudflare blocks it often enough that it is not reliable as a primary method.

CSV import. Letterboxd lets users export their data as CSV files (`ratings.csv`, `reviews.csv`, `watched.csv`). Parsing these gives the most complete history with no scraping involved, but it requires the user to go through the export process themselves.

3.2 Profile Discovery

We needed a set of active users to build our film corpus. We scraped the Letterboxd pages for a few popular recent movies (*Oppenheimer*, *Anora*, *The Substance*, etc.), pulled out reviewer usernames from the review links in the page HTML, and checked that each profile page loaded (HTTP 200). This gave us 100 confirmed active users.

3.3 TMDB Enrichment

For each film we query the TMDB API v3 detail endpoint with `append_to_response=keywords,credits` and grab:

- **overview:** plot summary (the main text we feed to SBERT)
- **genres:** genre labels
- **cast:** top 10 billed actors
- **director:** pulled from crew credits
- **keywords:** up to 15 thematic tags
- **vote_average** and **vote_count:** TMDB community rating and vote count

Most films already have a `tmdb_id` from the RSS feed, so we can fetch details by ID directly. For the few that don't, we fall back to searching by title + year, and retry without the year if nothing comes up. We cap API calls at 4 per second to stay under the rate limit.

3.4 Cross-User Aggregation

After scraping all 100 users, we merge their data into one corpus. Films are grouped by `tmdb_id` so each movie appears once, with all of its user reviews and ratings pooled together. We also store a pre-formatted `embedding_text` string (overview + genres + director + keywords) and some aggregate stats like mean rating and review count.

When the same film shows up in multiple users' diaries, we keep all their reviews instead of throwing out duplicates. At embedding time, we encode each review separately with SBERT and average the vectors. A film that 10 people reviewed ends up with a more stable embedding than one with a single review.

4 BACKGROUND: PCA

Principal Component Analysis (PCA) is a standard technique for reducing the number of dimensions in a dataset while keeping as much of the variance as possible. Given a matrix of n data points in d dimensions, PCA finds a new set of axes (the principal components) that are linear combinations of the original features, ordered by how much variance each one explains. The first component points in the direction of greatest variance, the second is orthogonal to the first and captures the next most variance, and so on.

To reduce dimensionality, we keep only the top k components and project every data point onto them, going from d dimensions down to k. The key assumption is that if a small number of components explains most of the variance, the remaining dimensions are mostly noise or redundancy and can be dropped without losing much information.

In our case, we start with 791-dimensional film vectors (two 384-d SBERT embeddings plus 23 metadata features). Many of these dimensions are correlated: a review of a horror film will mention similar things as the plot summary, so the content and review embeddings overlap. PCA lets us compress this down to 128 dimensions while retaining the structure that matters for measuring film similarity.

Before running PCA, we standardize each feature to zero mean and unit variance with `StandardScaler`. This is important because the SBERT dimensions are roughly unit-scale, but features like year and vote count are on completely different scales. Without standardization, PCA would be dominated by whichever features happen to have the largest raw values.

5 EMBEDDING CONSTRUCTION

We represent each film as a single vector made up of four groups of features. Before PCA, these add up to 791 dimensions:

5.1 Content Embedding (384 dimensions)

We concatenate the plot overview, genre names, director, and keywords into one string (`embedding_text`) and run it through the `all-MiniLM-L6-v2` sentence-transformer (22M parameters, 384-d output). This gives us a 384-dimensional vector for each film's content.

5.2 Review Embedding (384 dimensions)

Each user review for a given film is encoded with the same SBERT model, and we average all the review vectors element-wise to get one review embedding per film. Films with no reviews get a zero vector here.

We average in embedding space instead of concatenating the raw review texts. Concatenation would blow past the model's token limit for films with many reviews, and averaging gives each review equal weight no matter how long it is.

5.3 Scalar Features (4 dimensions)

- **Year:** release year, so the model can pick up on era preferences
- **Average user rating:** mean Letterboxd star rating across our scraped users
- **TMDB rating:** community vote average (1.0 to 10.0)
- **Log vote count:** log(1 + vote_count); we take the log so blockbusters with millions of votes don't dominate

5.4 Genre Multi-Hot (19 dimensions)

A binary vector with one bit per TMDB genre (19 total). We include this because the SBERT embedding sometimes blurs similar genres together (e.g. "thriller" and "horror"), and having explicit genre bits helps keep them distinct.

5.5 Dimensionality Reduction

We standardize all 791 features to zero mean and unit variance, then run PCA to bring it down to 128 dimensions. On our corpus, 128 components capture about 62.3% of the variance. PCA helps mainly because the content and review SBERT vectors overlap a lot for the same film (reviews tend to mention the plot), so there is a lot of redundancy to compress out. It also speeds up the nearest-neighbor search at recommendation time.

6 DATASET STATISTICS

Metric	Value
Users scraped	100
Total film entries (across all users)	4,852
Unique films (by TMDB ID)	2,565
Films with TMDB metadata	2,538 (98.9%)
Films with embedding text	2,542 (99.1%)
User ratings collected	4,054 (83.6%)
User reviews collected	3,206 (66.1%)
Films with multiple reviews	398 (15.5%)
Avg. reviews per multi-review film	4.7
Genres represented	19
Languages represented	37
Year range	1896–2026
Avg. films per user	48.5
Min / Max films per user	32 / 50

Table 1: Corpus statistics after scraping and enrichment.

7 RECOMMENDATION METHOD

Given a Letterboxd username, the system does the following:

(1) **Scrape** the user's RSS feed to get their recent watches, ratings, and reviews.
(2) **Look up** each film's pre-computed 128-d embedding from the corpus. If a film is not in the corpus, we encode it on the fly with the same pipeline.
(3) **Build a user profile vector** as a weighted average of their film embeddings:

$$\mathbf{u} = \frac{\sum_i r_i \cdot \mathbf{e}_i}{\sum_i r_i}$$

where r_i is the star rating for film i and $\mathbf{e}_i$ is its embedding. Films rated higher by the user have more influence on the profile.
(4) **Rank** every corpus film the user hasn't seen by cosine similarity to $\mathbf{u}$.
(5) **Return** the top-N results.

There is no training step. Everything runs against pre-computed embeddings, so it is fast. If we add a learned model later, this is the baseline.

8 EXAMPLE RECOMMENDATIONS

We picked three Letterboxd users with pretty different taste to see how the system handles each one. Table 2 lists their highest-rated films next to what our system recommended. Because we pull data from each user's RSS feed, which Letterboxd caps at 50 entries, these results are based on their 50 most recent diary entries rather than their full watch history (which may be much larger). Recommendations take under two seconds per user.

@davechen. favors violent, stylish genre films (Kill Bill, Hard Boiled, Oldboy). The recs match: Cool Hand Luke, A Clockwork Orange, and Magnolia all share that intense, director-driven quality.

@demyvarda. watches mostly French cinema, musicals, and romance. The recs stay in that lane: Princess Bride, Little Miss Sunshine, The Worst Person in the World. Nothing dark or action-heavy.

@broey_deschanel. has broad taste (documentary, Italian art cinema, indie drama), but the recs all focus on women's experiences: An Unmarried Woman, Late Autumn, Something's Gotta Give. The system picks up on theme over genre.

All three get completely different lists despite some overlap in what they've watched, since the rating-weighted profile emphasizes what each user likes most.

9 REPRODUCIBILITY

All code is in the project repository. To run the full pipeline from scratch:

```
pip install -r requirements.txt
cp .env.example .env # add TMDB API key

python scripts/scrape_batch.py
python scripts/build_corpus.py
python scripts/build_embeddings.py
python scripts/recommend.py --username <user>
```

10 REMAINING WORK

What we still need to finish before the end of the semester:

(1) **Formal evaluation.** Right now we only have the qualitative examples above. We need to split each user's history into train/test, build the profile from training, and see if the test films rank highly. We plan to report Recall@10 and MRR.
(2) **Expand the corpus beyond 50 films per user.** The RSS feed caps at 50 entries. We want to try the CSV export path or paginated HTML scraping to get full watch histories for at least some users and see if more data improves recommendations.
(3) **Baseline comparisons.** Compare SBERT + PCA against simpler methods like TF-IDF on plot text, genre-only matching, and popularity ranking. Need to check that the extra complexity actually helps.
(4) **Rating prediction model.** Train a small MLP or ridge regression that takes a user profile vector and a film embedding and predicts the star rating. This was in the original proposal and would go beyond just nearest-neighbor ranking.
(5) **Hyperparameter tuning.** Try different PCA dimensions (64, 128, 256), a bigger SBERT model (`all-mpnet-base-v2`), and different weighting schemes for the user profile vector.
(6) **User study.** Have a few people try it with their own Letterboxd accounts and get feedback on whether the recommendations are any good.
(7) **Clean up codebase.** Add a `README`, fill in missing type hints, and make sure everything runs end-to-end from a clean clone.

11 CONCLUSION

	@davechen *1,716 films total (50 most recent used)*	**@demyvarda** *2,861 films total (50 most recent used)*	**@broey_deschanel** *287 films total (50 most recent used)*
Top-rated films	Kill Bill: The Whole Bloody Affair (5.0) Hard Boiled (5.0) Oldboy (5.0) Nirvanna the Band the Show the Movie (5.0)	Call Me by Your Name (5.0) Twin Peaks: Fire Walk with Me (5.0) Donkey Skin (5.0) The Young Girls of Rochefort (5.0)	Nirvanna the Band the Show the Movie (5.0) If I Had Legs I'd Kick You (5.0) Stories We Tell (5.0) La Dolce Vita (5.0)
Top recs (sim.)	Cool Hand Luke (0.50) A Clockwork Orange (0.46) Heaven & Earth (0.45) Magnolia (0.44) Promising Young Woman (0.44)	The Princess Bride (0.41) Little Miss Sunshine (0.36) Wicked: For Good (0.35) Freakier Friday (0.38) The Worst Person in the World (0.35)	An Unmarried Woman (0.47) Sleepless in Seattle (0.46) Something's Gotta Give (0.46) A Summer Place (0.45) Late Autumn (0.42)

Table 2: Recommendations for three users based on their 50 most recent Letterboxd diary entries (the RSS feed limit). Top-rated films and recommendations are ranked by cosine similarity against the user's profile vector.